```
import A5O
set_option autoImplicit false

-- Sufficiency probe against A5O.lean v4 (SHA-256 0e5ebe3f…), unmodified.
-- Question: does O (via achievesADA) SUFFICE for accountability, where
-- accountability is written over what actually EXECUTES, not over what
-- the system merely enforces?

section Probe
variable {Principal Agent Action Time AoC : Type}
variable (C : Principal → Agent → Action → Time → Prop)
variable (I : Principal → Prop)
variable (M : Agent → Action → Prop)
variable (S : Agent → Action → Prop)
variable (R : Principal → Agent → Action → Time → Prop)
variable (Appointed : Principal → Agent → Action → Time → Prop)
variable (Executes : Agent → Action → Time → Prop)
variable (inAoC : Action → AoC → Prop)

-- Accountable: every execution in an area of consequence resolves to a
-- principal for whom O holds at that time. Written over Executes, in the
-- file's own primitives — never over sys.enforces, so it is not O → O.
def Accountable : Prop :=
  ∀ (A : Agent) (X : Action) (t : Time) (domain : AoC),
    inAoC X domain → Executes A X t →
      ∃ P : Principal, O C I M S R Appointed P A X t

-- Interception: nothing executes in an area of consequence unless the
-- system enforced it for some principal. This is the candidate "seventh
-- thing": it is NOT one of O's six conjuncts and NOT stated anywhere in v4.
def Intercepts (sys : ADASystem Principal Agent Action Time) : Prop :=
  ∀ (A : Agent) (X : Action) (t : Time) (domain : AoC),
    inAoC X domain → Executes A X t → ∃ P : Principal, sys.enforces P A X t

-- (1) WITH interception, sufficiency is a two-line theorem from achievesADA.
theorem sufficiency_with_interception
    (sys : ADASystem Principal Agent Action Time)
    (hADA : achievesADA C I M S R Appointed inAoC sys)
    (hInt : Intercepts Executes inAoC sys) :
    Accountable C I M S R Appointed Executes inAoC :=
  fun A X t domain hDom hExec =>
    let ⟨P, hEnf⟩ := hInt A X t domain hDom hExec
    ⟨P, hADA P A X t domain hDom hEnf⟩
```

```
-- (2) WITHOUT interception, sufficiency is FALSE: a concrete system that
-- achieves ADA per v4's definition (it never enforces anything at all, so
-- it vacuously never enforces outside O), yet an execution occurs in an
-- area of consequence with no principal satisfying O.
end Probe

section Countermodel
-- Everything Unit; one execution happens; O fails because nothing is Appointed.
def cmC : Unit → Unit → Unit → Unit → Prop := fun _ _ _ _ => True
def cmI : Unit → Prop := fun _ => True
def cmM : Unit → Unit → Prop := fun _ _ => True
def cmS : Unit → Unit → Prop := fun _ _ => True
def cmR : Unit → Unit → Unit → Unit → Prop := fun _ _ _ _ => True
def cmAppointed : Unit → Unit → Unit → Unit → Prop := fun _ _ _ _ => False
def cmExecutes : Unit → Unit → Unit → Prop := fun _ _ _ => True
def cmInAoC : Unit → Unit → Prop := fun _ _ => True

-- The silent system: enforces nothing. (A bearer system "enforces"
-- nothing in O's sense either — execution just happens.)
def SilentSystem : ADASystem Unit Unit Unit Unit := ⟨fun _ _ _ _ => False⟩

theorem silent_achieves_ADA :
    achievesADA cmC cmI cmM cmS cmR cmAppointed cmInAoC SilentSystem :=
  fun _ _ _ _ _ _ hEnf => hEnf.elim

theorem silent_not_accountable :
    ¬ Accountable cmC cmI cmM cmS cmR cmAppointed cmExecutes cmInAoC :=
  fun h =>
    let ⟨_, hO⟩ := h () () () () trivial trivial
    hO.2.2.2.2.2

-- Therefore: achievesADA alone does not imply Accountable.
theorem ADA_does_not_suffice :
    ∃ (sys : ADASystem Unit Unit Unit Unit),
      achievesADA cmC cmI cmM cmS cmR cmAppointed cmInAoC sys ∧
      ¬ Accountable cmC cmI cmM cmS cmR cmAppointed cmExecutes cmInAoC :=
  ⟨SilentSystem, silent_achieves_ADA, silent_not_accountable⟩

-- And the same silent system fails exactly Intercepts — the gap is
-- located, not merely asserted.
theorem silent_not_intercepts :
    ¬ Intercepts cmExecutes cmInAoC SilentSystem :=
  fun h => let ⟨_, hEnf⟩ := h () () () () trivial trivial; hEnf
end Countermodel
```

```
#print axioms sufficiency_with_interception
#print axioms ADA_does_not_suffice
#print axioms silent_not_intercepts
```